

PRIMG : Efficient LLM-driven Test Generation Using Mutant Prioritization

Mohamed Salah Bouafif
Polytechnique Montréal
Montréal, Canada
mohamed-salah.bouafif@polymtl.ca

Mohammad Hamdaqa
Polytechnique Montréal
Montréal, Canada
mhamdaqa@polymtl.ca

Edward Zulkoski
Quantstamp
California, USA
ed@quantstamp.com

Abstract

Mutation testing is a widely recognized technique for assessing and enhancing the effectiveness of software test suites by introducing deliberate code mutations. However, its application often results in overly large test suites, as developers generate numerous tests to kill specific mutants, increasing computational overhead. This paper introduces PRIMG (Prioritization and Refinement Integrated Mutation-driven Generation), a novel framework for incremental and adaptive test case generation for Solidity smart contracts. PRIMG integrates two core components: a mutation prioritization module, which employs a machine learning model trained on mutant subsumption graphs to predict the usefulness of surviving mutants, and a test case generation module, which utilizes Large Language Models (LLMs) to generate and iteratively refine test cases to achieve syntactic and behavioral correctness.

We evaluated PRIMG on real-world Solidity projects from Code4Arena to assess its effectiveness in improving mutation scores and generating high-quality test cases. The experimental results demonstrate that PRIMG significantly reduces test suite size while maintaining high mutation coverage. The prioritization module consistently outperformed random mutant selection, enabling the generation of high-impact tests with reduced computational effort. Furthermore, the refining process enhanced the correctness and utility of LLM-generated tests, addressing their inherent limitations in handling edge cases and complex program logic.

CCS Concepts

• **Security and privacy** → *Malware and its mitigation; Software security engineering; Artificial immune systems*; • **Software and its engineering** → *Application specific development environments*;

Keywords

Empirical Study, Smart Contracts, Mutation testing, Large Language Models, Test Generation

ACM Reference Format:

Mohamed Salah Bouafif, Mohammad Hamdaqa, and Edward Zulkoski. 2025. PRIMG : Efficient LLM-driven Test Generation Using Mutant Prioritization. In *Evaluation and Assessment in Software Engineering (EASE '25)*, June 17–20, 2025, Istanbul, Türkiye. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3756681.3756991>



This work is licensed under a Creative Commons Attribution 4.0 International License. *EASE '25, Istanbul, Türkiye*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1385-9/25/06
<https://doi.org/10.1145/3756681.3756991>

1 Introduction

Software testing is a critical component of the software development lifecycle, aimed at ensuring the quality and reliability of software systems. As systems grow in complexity, the need for effective and efficient testing strategies becomes increasingly paramount. One of the primary challenges in developing efficient test suites is achieving comprehensive coverage while minimizing redundancy. As software systems become more complex, the number of possible execution paths and scenarios increases exponentially. This makes it difficult to create test cases that adequately cover all aspects of the system without becoming unwieldy or time-consuming to execute [21]. To address these challenges, researchers and practitioners have turned to advanced techniques such as search-based software testing (SBST) and heuristic approaches. SBST applies optimization algorithms, such as genetic algorithms, to automatically generate test cases that maximize coverage and fault detection while minimizing the overall size of the test suite. These techniques can explore vast search spaces more efficiently than manual approaches, potentially uncovering edge cases and vulnerabilities that might otherwise be missed. While such approaches have proven their efficiency in generating test cases that uncover edge cases, they require significant computational resources, especially for large and complex systems. In recent years, mutation testing (MT) has emerged as a powerful technique for assessing and improving the quality of test suites while also guiding the generation of high-quality test cases for the program under test (PUT). Mutation testing is a software testing technique that involves modifying a program's source code to create faulty versions, known as mutants, to evaluate the effectiveness of a test suite in detecting errors. While the primary goal of mutation testing is to assess the quality of a test suite by determining whether it can identify and kill mutants, thereby ensuring the reliability and robustness of the software and unveiling edge cases, different approaches have been exploring the usage of surviving mutants to guide the generation of new test cases [28][8]. Generating an adequate test suite using mutation testing can be computationally expensive and time-consuming due to the vast number of possible mutants. Furthermore, using mutants as test goals can lead to an exponential increase in the size of the test suite since each mutant will be killed by a test, and thus running the test suite can become impractical for complex codebases. Liu et al. introduced AID [19], a method combining LLMs with differential testing to generate fault-revealing test cases for programs that have already passed traditional test suites. This approach demonstrates the potential of integrating advanced AI techniques with mutation testing to improve test case generation. However, the generation of valid and effective test cases remains a challenge. The authors in [27] highlighted issues with correctness in LLM-generated test

cases, while Yuan et al. proposed ChatTester[30], an approach that refines test cases iteratively to improve their quality. These studies underscore the need for sophisticated validation and refinement techniques when using AI-generated test cases.

While significant efforts have explored the use of Large Language Models (LLMs) for test case generation, several challenges have emerged, primarily related to ensuring the correctness of the generated tests and the computational intensity required to construct high-coverage test suites. Mutation testing aims to improve test quality and coverage by guiding developers to create tests targeting surviving mutants. These surviving mutants serve as test goals, with the objective of enhancing code coverage and identifying hidden vulnerabilities. However, as the complexity of software systems grows, the number of generated mutants increases exponentially, leading to scalability challenges in the process of test generation. Specifically, generating and validating test cases to kill a large number of mutants becomes a computationally expensive and time-consuming process, even when employing LLMs, which require additional resources for iterative refinement. Despite these scalability concerns, the state-of-the-art mutant selection technique remains random selection, which often fails to optimize computational efforts and test effectiveness [9, 17].

In this paper, we introduce PRIMG (Prioritization and Refinement Integrated Mutation-driven Generation), a novel approach tailored for incremental test suite generation. PRIMG leverages mutation testing to guide the effective use of LLMs in generating and refining test cases for Solidity smart contracts. Our approach aims to achieve effective test generation that optimizes the total number of generated tests while simultaneously enhancing the mutation score. Our proposed technique, employs two main components: a **mutation prioritization module**, responsible for selecting mutants as test goals for the generated test cases, and a **test case generation module**, which employs an LLM for generating and refining the test cases.

The rest of this paper is organized as follows. Section 2 briefly reviews the related works. Section 3 describes the different modules of our approach. We present our experimental setup, research questions, and experimental results in Section 4. We discuss our findings in Section 5. Threats to the validity of our results are reviewed in Section 6. Finally, we conclude the paper in Section 7, highlighting some avenues for future work.

2 Related Works

The proliferation of smart contracts has established them as a cornerstone of automation and trust, facilitating secure and transparent transactions across diverse industries, including finance, real estate, and supply chain management [5]. However, their increasing adoption has unveiled numerous challenges. The immutable and transparent nature of smart contracts paradoxically renders them susceptible to exploitation, misuse, and unforeseen vulnerabilities [33]. Consequently, the imperative to conduct rigorous audits of smart contracts arises, entailing a meticulous examination of their code and features to detect and mitigate potential risks[33]. The immutable nature of smart contracts adds a new layer of importance for developers and enhances the need for the development of a comprehensive and complete test suite [10].

Over the years, various tools have been developed to automate the task of test case generation for smart contracts. Numerous studies have explored the use of genetic algorithms for test case generation [7, 13]. For example, the authors of [23] investigated the application of meta-heuristic algorithms, implementing multiple search techniques such as random search and genetic algorithms (e.g., NSGAII[23], MOSA[23], and DynaMOSA[25]) for effective test case generation. While these approaches have demonstrated promising results, the genetic algorithm approach remains computationally expensive.

As highlighted in the in-depth study by Zou et al. [34], generating test suites for smart contracts is a complex task, with several challenges arising throughout their development and implementation. Among the surveyed developers, 54.7% reported the lack of mature and business-ready tools for blockchain-specific development. Additionally, these tools fail to address all corner cases and scenarios, which is the most critical challenge identified by the developers.

The emergence of large language models (LLMs) for software development tasks [12] has sparked interest in leveraging these models for test case generation [24]. Several tools and techniques have been proposed in this area. For example, Arghavan Moradi Dakhela et al. [4] suggested using mutation testing to guide LLMs in generating effective test cases. After an initial mutation campaign, their approach prompts the LLM with surviving mutants, requesting the generation of tests to kill the mutants. While their empirical evaluation showed promising results, the approach has yet to be tested on large datasets and real-world projects. Large codebases introduce three significant challenges: (1) the large number of generated mutants makes performing mutation testing computationally expensive and time-consuming, (2) the large number of tests required to achieve a high mutation score, and (3) real-world project requires intricate test setups and environments and more complex test case than simple assert-statement tests.

3 Approach

In this section, we discuss the different steps of our approach. Figure 1 shows an overview of our proposed approach, and Algorithm 1 presents the sequence of its different steps. PRIMG is composed of two main modules: **testcase generation module** and **mutants prioritization module**. We discuss each of the two modules in the following subsections.

3.1 Test Case Generation Module

Approach overview: This section details the process of generating test cases as outlined in Algorithm 1. Following an initial mutation testing phase, we identify and select the surviving mutants. The primary objective of the proposed approach is to develop unit tests capable of eliminating the maximum number of these surviving mutants. The process begins by selecting a specific mutant as the testing target, with the selection mechanism described in Section 3.2. The prioritized mutant is then used to generate an initial prompt for the large language model (LLM). Subsequently, the process enters a refinement phase, designed to ensure the correctness of the generated test cases, as elaborated in subsequent sections. If a test case is deemed correct, it is incorporated into the final test suite.

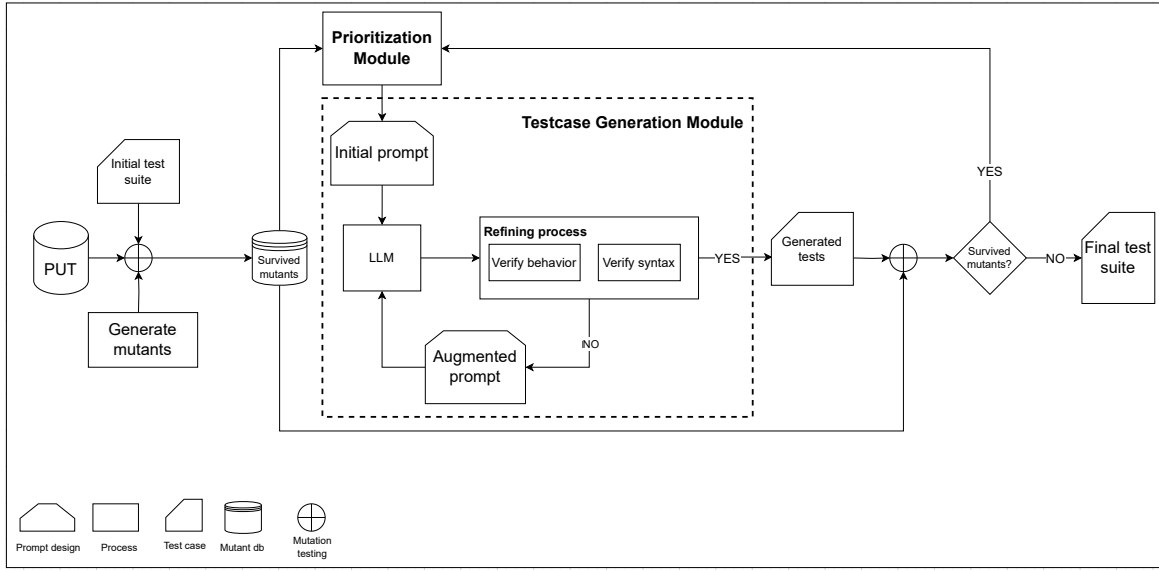


Figure 1: The proposed approach for generating a test case using LLMs.

Algorithm 1 PRIMG**Input:** *PUT*, *initialTestSuite*, *survivingMutants***Output:** *finalTestSuite*

```

1: prioritizedMutant ← PrioritizeMutant(survivingMutants)
2: initTest ← SelectTest(prioritizedMutant, initialTestSuite)
3: initPrompt ← CONCAT(initTest, prioritizedMutant, PUT)
4: rawIUT ← LLM(initPrompt)
5: FUT, success ← RefiningTest(rawIUT, PUT, prioritizedMutant)

6: if success then
7:   Add FUT to finalTestSuite
8: end if
9: return finalTestSuite

```

Conversely, if the refinement process fails to produce a correct test case, the task is classified as problematic or one for which PRIMG is unable to generate an appropriate test case.

3.1.1 Initial Prompt. Large Language Models (LLMs) are adept at performing tasks they have been trained on. However, fine-tuning LLMs to adapt to a new task is computationally expensive. Additionally, certain LLMs, such as Codex[3], excel in generating code but are closed-source, making fine-tuning for custom tasks infeasible.

Prompt-based learning [20, 31] offers an effective alternative for adapting LLMs to new tasks. A prompt is a combination of natural language and/or programming language context used as input to LLMs. Studies have demonstrated that incorporating natural language instructions as hints within prompts significantly enhances the ability of LLMs to perform novel tasks [22].

In our approach, the initial prompt consists of three key components:

Algorithm 2 RefiningTest**Input:** *rawIUT*, *PUT*, *prioritizedMutant***Output:** *FUT*, *error*

```

1: correctTest ← false
2: FUT ← rawIUT
3: counter ← 0
4: while not correctTest and counter < loopSize do
5:   syntaxVerif, errorSyntax ← Compile(FUT)
6:   behaviorVerif, errorBehavior ← RunTest(FUT, PUT)
7:   if syntaxVerif and behaviorVerif then
8:     correctTest ← true
9:   else
10:    augmentedPrompt ← CONCAT(FUT, PUT, errorSyntax,
11:      errorBehavior, prioritizedMutant)
12:    FUT ← LLM(augmentedPrompt)
13:    counter ← counter + 1
14:   end if
15: end while
16: return FUT, correctTest

```

- **The Program Under Test (PUT):** This represents the original program prior to mutation.
- **The Mutant Code:** This introduces modifications to the original program, representing potential faults or variations.
- **An Initial Test File:** This file is designed to validate the functionality and setup of the PUT within the testing framework. The test file is selected based on the following criteria:
 - (1) The test file should test functions defined in the PUT. This ensures the large language model (LLM) receives sufficient information about the testing environment and the imported libraries required to execute the tests.

- (2) The test file should contain diverse test cases. This enhances the LLM's understanding of the structure and variety of test cases, aiding in the generation of comprehensive and accurate tests.

3.1.2 Refining Process. As illustrated in Algorithm 2, after generating the raw Initial Unit Test (IUT), we employ a refining process to enhance its quality and effectiveness. The refining process consists of two primary steps:

Syntax Verifier and Behavior Verifier. In this step, we verify the correctness of the generated test case in two stages:

- (1) **Syntax Verification:** The test case is compiled, and logs are reviewed for syntax errors.
- (2) **Behavior Verification:** The test case is executed against the PUT. A valid test case must successfully pass when run on the PUT.

Prompt Augmentation. If the previous step fails (e.g., the test case fails to compile or the PUT fails when tested), we proceed to prompt augmentation. In this step:

- A new prompt is constructed using the PUT, the mutant code, the FUT, and the error logs from the previous step.
- This enhanced prompt is used to generate a refined test case, referred to as the Final Unit Test (FUT), which resolves the issues encountered in the initial iteration.

By iteratively refining and augmenting the prompt, we ensure the generation of high-quality test cases capable of validating both the syntax and behavior of the program under test.

3.2 Prioritization module

3.2.1 Dynamic Mutant Subsumption Graph. In mutation testing, a test set T is considered *mutation-adequate* with respect to a mutant set M if for every non-equivalent mutant $m_i \in M$, there exists a test $t \in T$ such that t detects m_i [16]. However, mutation operators generate a significantly larger number of mutants than necessary, leading to redundancy. This redundancy is formally captured by the concept of *minimal mutation*. A dominator set $D \subseteq M$ is a minimal subset of M such that any test set adequate for D is also adequate for M . A key aspect of mutation testing is the *subsumption relationship* between mutants. Given a finite set of mutants M and a finite test set T , a mutant m_i is said to *dynamically subsume* another mutant m_j if a test $t \in T$ detects m_i and every test that detects m_i also detects m_j . This relationship is *strict* if m_i subsumes m_j , but m_j does not subsume m_i . If two mutants m_i and m_j are detected by exactly the same tests in T , the subsumption is not strict [15]. The *Dynamic Mutant Subsumption Graph (DMSG)* captures these subsumption relationships. In a DMSG, each node represents a maximal set of redundant mutants, and edges represent the subsumption relationships between sets. Specifically, if mutant m_i strictly subsumes mutant m_j , there is a directed edge from the node containing m_i to the node containing m_j . Any test that detects a mutant in the DMSG is guaranteed to detect all mutants subsumed by that mutant, including those in the same node or lower in the graph. This structure provides an efficient way to identify and eliminate redundant mutants, thereby improving the mutation testing process and reducing the computational burden on developers.

3.2.2 Test Completeness Advancement Probability, TCAP. Test Completeness Advancement Probability (TCAP), as defined in [15], is employed to measure mutant usefulness in the context of test case generation. This measure quantifies the likelihood of a mutant, when presented as a test goal to a developer, eliciting a test case that enhances test completeness by killing the mutant and a maximum number of other mutants. This captures a key idea: a mutant is useful to be selected as the test goal, if when developing a test case that kills that mutant, other mutants are going to be killed by the same developed test case. This is due to the fact that the subsumed mutant, m_j is killed by the same tests that kills the dominant mutant m_i . In a subsumption relationship, the test case that kills the dominant mutant are also killing the subsumed mutant.

As discussed in [15], we define the usefulness of a mutant in terms of its capacity to detect (or kill) other mutants when it is itself killed. However, for surviving mutants (mutants not killed by the initial test suite), it is not feasible to construct a Dominance Mutation Subsumption Graph (DMSG). Instead, TCAP exploits the structural information of the DMSGs of killed mutants. By analyzing these graphs, we can identify mutants that are likely to be useful as test goals. The intuition is that the test suite developed to kill these selected mutants will also effectively kill other mutants, thereby improving the overall efficiency of the test generation process and optimizing the total number of needed tests. This approach builds on foundational ideas of mutation analysis and mutant subsumption as explored in prior research [14, 16]. By leveraging the dominance relationships and subsumption hierarchies among mutants, we aim to streamline the test case generation process, improving both coverage and effectiveness through the usage of a supervised machine learning model.

The usefulness factor of a mutant is defined as measurement of its ability to reveal additional mutants when used as a target for test case generation. The usefulness factor of a mutant has two properties:

- Dominator mutants, have usefulness factor equal to 1: By definition, each test that kills a dominator mutants kills every other subsumed mutant.
- Subsumed mutants, have a usefulness factor less than 1 and greater than 0

3.2.3 Machine learning module.

- **context feature:** Building upon the modeling approach and the feature set proposed by Samuel J. Kaufman et al. [15], we incorporated program context features extracted from a program's Abstract Syntax Tree (AST). For a given mutated AST node, we examined its syntactic context, including parent and child nodes within the AST as well as nesting details, and its semantic context, such as the data types of expressions, operands, and method parameters. It is important to highlight that previous studies [32] utilized information derived from test execution data, such as code coverage, to make predictions in the context of mutation analysis. As done in [15], our approach aims to identify mutants that are beneficial for generating test cases that have not yet been created, aligning with the goals of mutation testing.

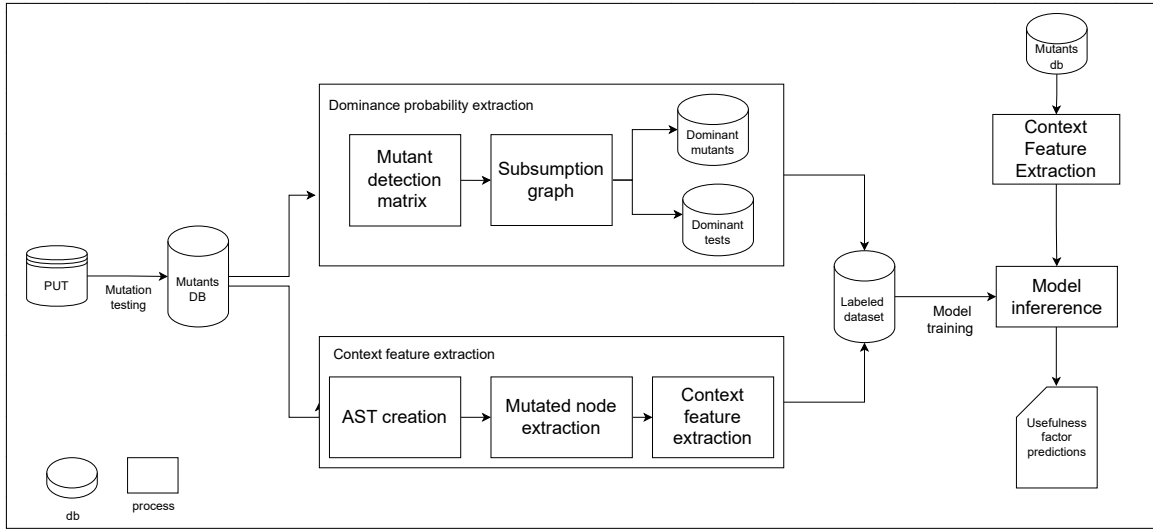


Figure 2: Overview of the dataset labeling process

- **Feature extraction and dataset labeling:** Figure 2 shows an overview of our dataset labeling approach and model training. Since we can only construct the DMSG for the killed mutants, only the killed mutant will be used for training the model.

After an initial mutation testing campaign using the initial test suite of the PUT, we create our labeled dataset. The labeling of the dataset is composed of two modules: Dominance probability extraction and Context feature extraction. The first component is the dominance probability extraction components which aims to label the killed mutants with an initial usefulness factor. Given the mutation testing results, we create the mutation matrix which maps test cases to mutants, indicating which test cases can detect (kill) or not specific mutants. We use the mutation matrix to construct the DMSG of mutants relative to the initial test suite as defined in the previous section. We extract the parent node which contains the dominant mutants. Those identified dominant mutants are labeled as useful having the useful factor equal to 1. The other mutants are labeled as having a useful factor equal to zero. The second component is context feature extraction module which aims to extract the context features from the Abstract Syntax Tree (AST) described in the previous section. First we start by comparing the AST of the mutant with the one of the PUT. We identify the mutated node. Using the mutated node we extract the remaining context features of the mutated node and its parent node context features. using the usefulness factor and the context features, we create the labeled dataset that will be used for training and inference.

- **Machine learning models:** We evaluated a small number of model design choices and training settings against an intrinsic measure of model performance with the ultimate goal of choosing a single model for downstream evaluation. Each training setting was a kind of hold-one-out train-test

split, using each held out project (or smart contract in a given project) at a time as the evaluation set. We trained all models using Scikit-Learn [26] and evaluated every combination of the following choices:

We have chosen these configuration:

- **Model choice :** We compared ridge regression to a random forest regression (max depth of 3 and 10 trees). [15, 18] .
- **features set :** We used a comprehensive set of features (All features, enumerated in [14])
- **Training Setting :** We evaluated the following train-test splits:
 - * *All Projects:* training on mutants from all projects, including those in non-held-out smart contracts in the same project, and testing on mutants in the held-out smart contracts;
 - * *Project-Only:* training on mutants in all but one held-out smart contract in a single project and testing on mutants in the held-out smart contract .

Building on findings in [15], we choose the following configuration for the experiments in the evaluation section:

Model=Linear, Features=All, Training Setting=Project-Only

4 Evaluation

In this section, we describe the evaluations we designed and conducted to investigate the following research questions.

- (1) **RQ1. How efficient is the pipeline for test case generation?**
- (2) **RQ2. How useful is the prioritization technique compared to random for efficient mutation testing?**

4.1 Experimental Setup

In this section, we present our experiment setup. Specifically, we describe the mutant generation tool, clarify the LLM of PRIMG and its setup, and indicate the benchmark data sets used in our

experiments. We conducted the experiment on an EC2 instance of type g5.xlarge with 26 GiB of GPU memory.

4.1.1 Experiment Parameters. We call the refining process on the LLM up to 10 times and collect the output that meets two criteria: The generated test must compile correctly and then pass the PUT. If after 10 runs, the LLM is not able to generate an output that satisfies these two conditions, we consider the mutant as a problematic task or a task for which PRIMG is not able to generate a test case.

For the prioritization module, for each project we train a machine learning model to predict the survived mutant usefulness factor. As mentioned in section 3.2.3, we use the configuration of: Linear model with all features. As we have already explained, the model is trained on mutants that have already been killed.

We use llama 3.1 as the large language model for the generation of test case and refinement of test cases.

4.1.2 Mutant Generator. To apply mutation testing, we need to generate different mutant versions of a PUT by modifying different lines of code to introduce vulnerabilities. For this purpose we use SuMo [1] as mutant generation tool. SuMo is a mutation testing tool adapted for smart contracts of Solidity. It uses different mutation operators to generate mutants including 25 Solidity-specific operators and 19 general-purpose mutation operators.

For testing, we used Ganache¹ as a local blockchain network for testing and deploying smart contracts and executing transactions for mutants and PUT. We use hardhat as testing framework².

4.1.3 Data sources. To conduct our experiments, we chose real world projects that we extracted from code4arena³. Code4arena is a platform facilitating decentralized security audits of smart contracts through competitive audit contests conducted by a global community of security researchers. We selected three different projects. The details of the selected project is in the following Table 1. To ensue rigor, we chose projects meeting these specific criteria:

- **Number of smart contracts:** this criterion ensures the selection of complex projects by focusing on those with more than 10 smart contracts, avoiding overly straightforward examples.
- **Initial test suite:** in the selected project, the initial test suite must have at least one test case testing each smart contract.
- **Mutation score:** projects are chosen with an initial mutation score (MS) below 80%, ensuring that the existing test suite is insufficient and requires additional test cases to improve coverage.

All scripts and dataset can be found here⁴.

4.2 Empirical Study Results

This section discusses our findings for each research question (RQ), focusing on the performance and efficiency of the proposed approaches.

4.2.1 Evaluation of the Performance of the Test Case Generation Module.

In this subsection, we target to answer the first research question. We divide the RQ1 to two sub questions and answer each one of them presenting our findings.

RQ1. How efficient is the pipeline for test case generation?

In this research question, we evaluate the performance of the test case generation module with respect to two key aspects: (1) the effectiveness of refining module in generating correct test cases and (2) the ability of these generated tests to kill new mutants that were not initially addressed by the original test suite. To ensure a comprehensive analysis, RQ1 is divided into the following subquestions:

- **RQ1.1:** How efficient is the pipeline for generating syntactically and behaviorally correct tests?

The objective here is to assess the accuracy of the test cases generated by the LLM. Correctness is evaluated on two dimensions: **syntactic correctness:** The test case compiles successfully, and **behavioral correctness:** The test case passes the original Program Under Test (PUT) without errors.

- **RQ1.2:** What is the impact of the size of the refining loop on the correctness of the generated tests?

This sub-question investigates how the number of iterations in the refining loop impacts the quality of the generated test cases. Specifically, it examines whether iterative improvements significantly enhance the syntactic and behavioral correctness of the test cases.

To answer RQ1.1, test cases were generated by prompting the LLM with and without the refining process using different loop sizes. A test case is considered correct if it successfully compiles and passes the original PUTs. The results were collected from three projects, as listed in Table 1.

To answer RQ1.1 and statistically evaluate the impact of the refining process, we formulated the following hypotheses [29]:

RQ1.1-H0: *The refining process does not significantly improve the number of correct tests compared to a single-shot prompt.*

RQ1.1-H1: *The refining process significantly improves the number of correct tests compared to a single-shot prompt.*

To evaluate the efficiency of the test generation process, we analyzed the performance of test case generation with respect to the number of correctly generated test cases across varying trial counts. Table 3 summarizes the results obtained when running the process with 1 trial, 5 trials, and 10 trials. To assess the statistical significance of the differences between these configurations, we performed pairwise Z-tests for proportions, a statistical method suitable for comparing success rates across independent trials.

The comparisons were conducted across three projects: Allobase, Particle, and Quadrata. The results of the Z-tests for each project are as follows:

- **Allobase:** Significant differences were observed between 1 trial and 5 trials ($Z = -19.08, p < 0.05$) as well as 1 trial and 10 trials ($Z = -19.20, p < 0.05$). No significant difference was found between 5 trials and 10 trials ($Z = -0.22, p > 0.05$).

¹Truffle Suite, *Ganache - One Click Blockchain*, <https://trufflesuite.com/ganache/>, accessed on Nov 26, 2024.

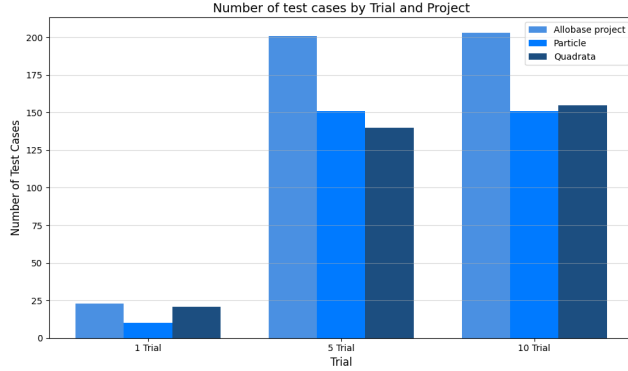
²Hardhat, <https://hardhat.org/>, accessed on Nov 26, 2024.

³<https://code4arena.com>

⁴<https://github.com/Salah-SH/llm-mutation-testing>

Table 1: Data sources

Project Id	# smart contracts	# initial test files	Initial mutation score %
Allobase	31	12	35.89
Particle	19	7	46.86
Quadrata	10	9	77

**Figure 3: Number of test cases by trial and projects**

- **Particle:** Significant differences were observed between 1 trial and 5 trials ($Z = -12.13, p < 0.05$) and 1 trial and 10 trials ($Z = -12.13, p < 0.05$). No significant difference was found between 5 trials and 10 trials ($Z = 0.0, p = 1.0$).
- **Quadrata:** Significant differences were observed between 1 trial and 5 trials ($Z = -10.24, p < 0.05$) and 1 trial and 10 trials ($Z = -10.54, p < 0.05$). No significant difference was found between 5 trials and 10 trials ($Z = -0.35, p = 0.73$).

Project	Single Shot	Refining process loop of 5	Refining process loop of 10
Allobase	23/500	201/500	203/500
Particle	10/500	151/500	151/500
Quadrata	21/500	140/500	145/500

Table 2: Number of correct test case generated in different Projects using different loop size

Specifically, for the projects Allobase, Particle, and Quadrata, the Z-test results provide strong evidence that increasing the number of trials significantly enhances the success rates of generating correct tests. These findings suggest that using more trials contributes to better test generation performance, as evidenced by the statistically significant improvements observed when increasing from 1 trial to 5 trials and from 1 trial to 10 trials. However, no significant differences were found between 5 and 10 trials across all projects, indicating diminishing returns beyond 5 trials. The observed differences in success rates are statistically significant under the experimental conditions and datasets analyzed. Thus, increasing the number of trials, particularly from 1 to 5, outperforms single-shot attempts in generating correct tests, highlighting the value of incorporating multiple trials in improving LLM-generated test cases.

Additionally, as shown in Table 2, the impact of the refining process on the performance of test case generation is evident. In the *Allobase* project, the single-shot approach generates only 4.6% of correct tests, whereas using the refining process with a loop size of 5 iterations results in a substantial improvement, with 40.2% of tests being correct. Similarly, for the *Particle* project, the single-shot approach yields only 2% correct tests, while applying the refining process with 5 iterations increases the correct tests to 30.2%. In the case of *Quadrata*, the single-shot approach results in 4.2% correct tests, and refining with 5 iterations improves this to 28%. These results indicate a significant performance boost when employing the refining module, demonstrating that iterative refinement is a crucial factor in improving the accuracy and effectiveness of the test case generation module. Therefore, we conclude that utilizing the refining module greatly enhances the performance of the test case generation process, making it a valuable strategy for generating higher-quality test cases in all evaluated projects.

Finding 1: The refining module can significantly improve the correctness of the generated tests.

To answer **RQ1.2**, we conducted an additional analysis focusing on the effect of loop size on the correctness of the generated tests. In this research question, our primary objective is to evaluate the role of the number of iterations (loop size) within the refining module on the overall performance of the test case generation process. The hypothesis guiding this investigation was based on the premise that increasing the number of loops could lead to more refined and accurate test cases. The results of this analysis, including detailed performance comparisons for each project, are presented in Table 2.

We formulated the null hypothesis (H_0) and alternative hypothesis (H_1) to test the impact of the number of trials as follows:

RQ1.2-H0: Both 5 and 10 loops significantly improve the number of correct test cases compared to single-shot for all projects.

RQ1.2-H1: Increasing the number of trials in the test generation process positively impacts the success rate of generated test cases.

To analyze the performance of the test generation module across different trial configurations (1 trial, 5 trials, and 10 trials), we employed pairwise Z-tests for proportions. These tests compare the success rates of correctly generated tests for different numbers of trials. The Z-test was applied to all three projects, Allobase, Particle, and Quadrata, and the results are summarized below:

- **Allobase:** No significant difference was found between 5 and 10 trials ($Z = -0.22, p > 0.05$).
- **Particle:** No significant difference was found between 5 and 10 trials ($Z = 0.00, p > 0.05$).

- **Quadrata:** No significant difference was found between 5 and 10 trials ($Z = -0.35, p > 0.05$).

These findings suggest that increasing the number of trials from 1 to 5 significantly improves the success rates of test case generation across all projects. However, further increases beyond 5 trials do not result in statistically significant improvements. The Z-test results emphasize the value of employing multiple trials, particularly up to 5, to enhance the accuracy and reliability of LLM-generated test cases.

Finding 2: No significant improvement is observed when increasing from 5 to 10 loops, suggesting diminishing returns beyond 5 iterations.

Overall, the results suggest that the refining loop technique using a loop size of 5 is the best configuration for optimal test case generation performance.

4.2.2 Evaluation of the performance of prioritization module. In this subsection, we target the second research question investigating the impact and the effectiveness of the prioritization module for improving the efficiency of the generated tests.

RQ2. How useful is the prioritization technique compared to random for efficient mutation testing?

To evaluate the prioritization module, we perform analysis on the tests generated for the projects as mentioned in Section ?? . For each project among the three projects, we generated 500 tests using the test case generation module. We train a linear model for each project as described in Section 3.2. We use the configuration *Model=Linear, Features=all, Training Setting=Project-Only*.

We distinguish between 2 types of tests: (1) Prioritized test is a test case that have been generated targeting a prioritized mutant and (2) random test is test that has been generated targeting a random mutant.

In Table ?? we illustrate the results of the mutation testing we executed using three random set of tests composed of 50 tests and one prioritized set of tests. To answer RQ2, we conducted a hypothesis test on the results project Allobase, Particle and Quadrata as presented in Table ?? . We formulated the null hypothesis (H_0) and the alternative hypothesis (H_1) as follows:

RQ2- H_0 : *There is no statistically significant difference in the performance of prioritized generated tests compared to randomly generated tests*

RQ2- H_1 : *There is a statistically significant difference in the performance of prioritized generated tests compared to randomly generated tests*

Similar to RQ1.1 and RQ1.2, we compare the efficacy of these strategies (i.e., prioritized tests vs. randomly selected tests) by using Z-test.

- **Allobase:** The Z-test for this comparison yielded $Z = -15.98, p < 0.05$, indicating a statistically significant difference between the two methods, with Method 2 outperforming Method 1.

- **Particle:** The Z-test for this comparison yielded $Z = -12.94, p < 0.05$, also showing a statistically significant difference between the two methods, with Method 2 performing better than Method 1.

- **Quadrata:** The Z-test for this comparison yielded $Z = -14.12, p < 0.05$, demonstrating a statistically significant difference between the two methods, with Method B outperforming Method A.

These results suggest that, across all three projects, Allobase, Particle, and Quadrata, the test suite composed of prioritized tests significantly outperforms the test suite composed of random tests set generated using random selection of mutants in killing mutants. The observed differences in success rates are statistically significant under the experimental conditions analyzed, highlighting the effectiveness of the prioritization technique in killing mutants. Thus, the prioritization module consistently enhances test performance, demonstrating its superiority over random selection in all the evaluated projects. These results highlight the potential of the prioritization strategy as a valuable tool in mutation testing, contributing to more efficient and targeted test generation in software testing practices.

Finding 3: There is a statistically significant difference in the performance of prioritized generated tests compared to randomly generated tests in killing mutants.

5 Discussions

5.1 Interpretation of the importance of the loop size on the effectiveness of test case generation

Although the refining module have shown impressive results in generating and refining generated tests, experiments in the evaluation section shows that increasing the size of the loop does not considerably improve the number of correctly generated tests since even having a larger trial number, the LLM still fails to generate correct tests no matter the number of repetitions. In fact, LLMs operate based on statistical patterns in their training data rather than true understanding of programming concepts or software semantics[6]. This can lead to what researchers call "hallucination" - generating plausible-looking but incorrect code that doesn't actually fix the underlying bug[11]. As Dou et al. observed in their extensive study, functional bugs constitute the highest proportion of errors in LLM-generated code fixes, with misunderstanding and logic errors being particularly prevalent[6]. Even after multiple attempts, an LLM may continue to make similar mistakes if it fundamentally misunderstands the problem or lacks the specific knowledge needed to generate a correct fix. Besides, the token-by-token generation process of LLMs can lead to inconsistencies in longer code sequences, even if individual parts seem correct. This is particularly problematic for bug fixes that require changes across multiple lines or functions. The authors in [6] observed that LLMs tend to perform better on localized, single-line fixes but struggle with bugs that require broader context or multiple coordinated changes. Furthermore, the limited context window of most LLMs can prevent them

Project	Randomly Selected Tests			Prioritized Tests	Ground Truth
	1	2	3		
Allobase	200	158	8	300	465
Particle	196	54	180	250	350
Quadrata	55	205	200	256	403

Table 3: Overlaid Table representing the number of killed mutants using Randomly Selected Tests, Prioritized Test, and Ground Truth

from fully considering all relevant parts of a large codebase, leading to fixes that may solve one issue but introduce new bugs elsewhere. These structural limitations of LLMs mean that even with repeated attempts, they may struggle to generate consistently correct and comprehensive bug fixes for more complex software issues.

5.2 Adequacy of our approach for practical applications

The PRIMG framework presents a practical and efficient solution for test suite generation by integrating automated test case generation with an advanced prioritization module. Leveraging Large Language Models (LLMs), the framework automates the creation of diverse, high-quality test cases, minimizing the manual effort required from developers while ensuring robust coverage of edge cases and complex scenarios. Complementing this, the prioritization module employs machine learning to rank and focus on the most impactful mutants, optimizing the testing process by reducing redundant efforts and maintaining high mutation coverage with a smaller, more efficient test suite. This dual approach significantly enhances the efficiency of mutation testing, streamlining the traditionally resource-intensive process and enabling developers to concentrate on critical tests. Particularly in domains like smart contract development, where rapid and reliable testing is crucial, PRIMG demonstrates its value by reducing computational overhead and improving the precision and practicality of the testing workflow. Its combination of automation, prioritization, and scalability establishes PRIMG as a powerful tool for advancing software quality in both research and industrial contexts.

6 Data Availability

All the scripts and results that have been used during our experiments can be found in this github repository <https://github.com/Salah-SH/llm-mutation-testing>

7 Threat to Validity

In this study, we employed two different prompt-based learning techniques: one-shot and prompting augmenting technique. However, we did not explore the potential impact of altering the natural language instructions or demonstrative examples (for the prompt augmenting technique) within our prompts. Modifying these instructions or utilizing different demonstrative examples more closely aligned with the PUT's functionality could potentially enhance the results.

A potential threat to the validity of our study arises from the choice of mutation operators defined within the SuMo framework[2]. Mutation operators play a critical role in generating test cases, and the limited set of operators available in SuMo

may not adequately represent all possible vulnerabilities in smart contracts. This can lead to a biased set of mutants, limiting the effectiveness of the generated tests in detecting a broad range of issues. If the operators do not cover diverse attack vectors or edge cases, the results of the mutation testing may not fully reflect the real-world security risks present in smart contracts, potentially compromising the generalize-ability of our findings.

Another validity threat arises from the exclusive use of Hardhat as the testing framework. While Hardhat is a widely adopted and robust tool for smart contract development and testing, its exclusive use may restrict the applicability of our results to other testing environments. Variations in how different frameworks execute and interact with smart contracts could lead to discrepancies in test outcomes. As a result, the conclusions drawn from Hardhat-based testing might not extend to projects using alternative frameworks, thus reducing the external validity of our study.

Additionally, this study did not evaluate the number of newly killed mutants or the impact of the new tests on the overall mutation score. Although this metric is important, our primary focus was on generating correct and valid test cases.

A further threat to validity arises from the practical challenges and resource limitations encountered during testing. While there is no inherent limitation in the availability of projects, the complexities involved in setting up the testing environment and the constrained resources for conducting extensive experiments restricted the scope of our analysis.

8 Conclusion

This paper presented PRIMG (Prioritization and Refinement Integrated Mutation-driven Generation), a novel framework for incremental and adaptive test case generation tailored for Solidity smart contracts. By integrating a mutation prioritization module with a test case generation module leveraging Large Language Models (LLMs), PRIMG addresses key challenges in mutation testing, such as excessive test suite sizes and inefficiencies in test generation. The mutation prioritization module employs machine learning models trained on mutant subsumption graphs to predict the usefulness of mutants, enabling the selection of high-impact test goals. Simultaneously, the test case generation module iteratively refines LLM-generated tests to ensure syntactic and behavioral correctness.

Our evaluation on real-world Solidity projects demonstrated PRIMG's effectiveness in improving mutation scores while reducing test suite size and computational overhead. The prioritization module consistently outperformed random mutant selection by enabling more efficient and targeted test case generation. Moreover, the refining process successfully mitigated common issues associated with LLM-generated tests, such as syntactic errors and limited bug-detection capabilities.

Although the current version of PRIMG employs mutant prioritization and LLMs to generate test cases for Solidity contracts programs, its design and evaluation methodology are fundamentally adaptable to various programming languages and models. Therefore, as future work, it can be easily expanded to encompass other programming languages or incorporate new LLMs.

References

- [1] Morena Barboni, Andrea Morichetta, and Andrea Polini. 2021. SuMo: A Mutation Testing Strategy for Solidity Smart Contracts. In *2021 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, Piscataway, NJ, USA, 50–59. doi:10.1109/AST52587.2021.00014
- [2] Morena Barboni, Andrea Morichetta, and Andrea Polini. 2022. SuMo: A mutation testing approach and tool for the Ethereum blockchain. *Journal of Systems and Software* 193 (2022), 111445. doi:10.1016/j.jss.2022.111445
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* N/A, N/A (2021), 10 pages. arXiv preprint, no volume or number applicable.
- [4] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C Desmarais. 2024. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology* 171 (2024), 107468.
- [5] TJ De Graaf. 2019. From old to new: From internet to smart contracts and from people to smart contracts. *Computer law & security review* 35, 5 (2019), 105322.
- [6] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, et al. 2024. What's Wrong with Your Code Generated by Large Language Models? An Extensive Study. *arXiv preprint arXiv:2407.06153* X, Y (2024), xx–yy.
- [7] Stefan Driessen, Dario Di Nucci, Geert Monsieur, Damian A Tamburri, and Willem-Jan van den Heuvel. 2021. Automated test-case generation for solidity smart contracts: The AGSolT approach and its evaluation. *arXiv preprint arXiv:2102.08864* XX, YY (2021), xx–yy.
- [8] Hang Du, Vijay Krishna Palepu, and James A Jones. 2024. Leveraging Propagated Infection to Crossfire Mutants. *arXiv preprint arXiv:2411.09846* N/A, N/A (2024), xx–yy.
- [9] Rahul Gopinath, Mohammad Amin Alipour, Iftekhar Ahmed, Carlos Jensen, and Alex Groce. 2016. On the limits of mutation reduction strategies. In *Proceedings of the 38th international conference on software engineering*. ACM, Austin, TX, USA, 511–522.
- [10] Havva Gulay Gurbuz, Bedir Tekinerdogan, Cagatay Catal, and Nagehan Pala Er. 2024. Test suite assessment of safety-critical systems using safety tactics and fault-based mutation testing. *Cluster Computing* XX, YY (2024), 1–25.
- [11] Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. 2024. A deep dive into large language models for automated bug localization and repair. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1471–1493.
- [12] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* XX, YY (2023), xx–yy.
- [13] Shunhui Ji, Shaoqing Zhu, Pengcheng Zhang, Hai Dong, and Jianan Yu. 2022. Test-case generation for data flow testing of smart contracts based on improved genetic algorithm. *IEEE Transactions on Reliability* 72, 1 (2022), 358–371.
- [14] René Just, Bob Kurtz, and Paul Ammann. 2017. Inferring mutant utility from program context. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, New York, NY, USA, 284–294.
- [15] Samuel J Kaufman, Ryan Featherman, Justin Alvin, Bob Kurtz, Paul Ammann, and René Just. 2022. Prioritizing mutants to guide mutation testing. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, New York, NY, USA, 1743–1754.
- [16] Bob Kurtz, Paul Ammann, Marcio E Delamaro, Jeff Offutt, and Lin Deng. 2014. Mutant subsumption graphs. In *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation Workshops*. IEEE, IEEE, New York, NY, USA, 176–185.
- [17] Bob Kurtz, Paul Ammann, Jeff Offutt, Márcio E Delamaro, Mariet Kurtz, and Nida Gökçe. 2016. Analyzing the validity of selective mutation with dominator mutants. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 571–582.
- [18] Robert G Kurtz Jr. 2018. *Improving mutation testing with dominator mutants*. Ph.D. Dissertation. George Mason University.
- [19] Kaibo Liu, Yiyang Liu, Zhenpeng Chen, Jie M Zhang, Yudong Han, Yun Ma, Ge Li, and Gang Huang. 2024. LLM-Powered Test Case Generation for Detecting Tricky Bugs. *arXiv preprint arXiv:2404.10304* N/A, N/A (2024), N/A.
- [20] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *Comput. Surveys* 55, 9 (2023), 1–35.
- [21] Phil McMinn. 2011. Search-based software testing: Past, present and future. In *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. IEEE, IEEE, New York, NY, USA, 153–163.
- [22] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-based prompt selection for code-related few-shot learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, Melbourne, Australia, 2450–2462.
- [23] Mitchell Olsthoorn, Dimitri Stallenberg, Arie Van Deursen, and Annibale Panichella. 2022. Syntest-solidity: Automated test case generation and fuzzing for smart contracts. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. ACM, Pittsburgh, PA, USA, 202–206.
- [24] Wendküni C Ouédraogo, Kader Kaboré, Haoye Tian, Yewei Song, Anil Koyuncu, Jacques Klein, David Lo, and Tegawendé F Bissyandé. 2024. Large-scale, Independent and Comprehensive study of the power of LLMs for test case generation. *arXiv preprint arXiv:2407.00225* 10 (2024), XX pages. arXiv preprint, no volume or number applicable.
- [25] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. 2018. Automated Test Case Generation as a Many-Objective Optimisation Problem with Dynamic Selection of the Targets. *IEEE Transactions on Software Engineering* 44 (2018), 122–158. <https://api.semanticscholar.org/CorpusID:3738201>
- [26] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in Python. *the Journal of machine Learning research* 12 (2011), 2825–2830.
- [27] Hamed Taherkhani and Hadi Hemmati. 2024. VALTEST: Automated Validation of Language Model Generated Test Cases. *arXiv:2411.08254 [cs.SE]* <https://arxiv.org/abs/2411.08254>
- [28] Frank Tip, Jonathan Bell, and Max Schäfer. 2024. LLMorpheus: Mutation Testing using Large Language Models. *arXiv preprint arXiv:2404.09952* N/A, N/A (2024), xx–yy.
- [29] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, Anders Wesslén, et al. 2012. *Experimentation in software engineering*. Vol. 236. Springer, Berlin.
- [30] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No more manual tests? evaluating and improving chatgpt for unit test generation. *arXiv preprint arXiv:2305.04207* n/a (2023).
- [31] Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing bugs in python assignments using large language models. *arXiv preprint arXiv:2209.14876* 10 (2022), xx–yy.
- [32] Jie Zhang, Ziyi Wang, Lingming Zhang, Dan Hao, Lei Zang, Shiyang Cheng, and Lu Zhang. 2016. Predictive mutation testing. In *Proceedings of the 25th international symposium on software testing and analysis*. ACM, New York, NY, USA, 342–353.
- [33] Liyi Zhou, Xihan Xiong, Jens Ernstberger, Stefanos Chaliasos, Zhipeng Wang, Ye Wang, Kaihua Qin, Roger Wattenhofer, Dawn Song, and Arthur Gervais. 2023. Sok: Decentralized finance (defi) attacks. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, IEEE, Piscataway, NJ, USA, 2444–2461.
- [34] Weiqin Zou, David Lo, Pavneet Singh Kochhar, Xuan-Bach Dinh Le, Xin Xia, Yang Feng, Zhenyu Chen, and Baowen Xu. 2021. Smart Contract Development: Challenges and Opportunities. *IEEE Transactions on Software Engineering* 47, 10 (2021), 2084–2106. doi:10.1109/TSE.2019.2942301